操作系统期中复习提纲

系统结构·进程线程·调度·同步·死锁·IPC

2026 年 10 月 26 日

目录

复习路线与课件索引 **1**

1. 操作系统结构、特权级与系统调用 **2**

2. 进程、地址空间与上下文切换 **4**

3. 线程模型、并发与并行 **6**

4. CPU 调度：比较、计算与权衡 **8**

5. 临界区、锁与原子操作 **10**

6. 死锁、安全状态与银行家算法 **12**

7. 信号量、条件变量、管程与屏障 **14**

8. 进程间通信：机制选择与异常处理 **17**

9. 最容易混淆的概念：闭卷对照 **19**

10. 自测题与核对答案 **20**

11. 三轮复习安排与考前检查 **21**

复习路线与课件索引

范围按已确认的 `OS-course` 中全部 10 份课件整理，共 378 页。页码均指 **PDF 文件从第 1 页起的实际页序**，不是幻灯片中偶尔保留的旧页脚编号。正文中的 I1、P1、L1 等简称对应下表。

“课内”是课件已有内容；“辨析”用于消除常见误解；“小拓展”是在原内容上多走一步。复习优先级按知识依赖和练习价值安排，不代表老师公布的题目或分值分布。没有把课程简介中提到、但本批课件尚未展开的虚拟内存算法、页置换、文件系统等内容加入主复习范围。

简称	原课件	页数	本提纲对应内容
I1	Intro01：课程简介	50	OS 的作用、发展与系统设计权衡
I2	Intro02：结构与组成	46	启动、特权级、异常、系统调用、内核架构
P1	Proc01：进程	36	地址空间、PCB、状态转换、进程 API
P2	Proc02：线程	35	TCB、线程模型、并发与并行
P3	Proc03：CPU 调度	34	调度指标、各类算法、实时调度
L1	Sync01：临界区与锁	46	竞态、软件互斥、原子指令、阻塞与唤醒
L+	Sync01：锁扩展	9	读写锁、RCU
D	Sync02：死锁	37	必要条件、安全状态、银行家算法、预防
S	Sync03：同步机制	44	P/V、条件变量、管程、屏障
C	Sync04：进程间通信	41	信号、管道、消息队列、共享内存、Socket

建议先理解这条主线：

```
OS 抽象与隔离
  → 进程表示资源，线程表示执行流
  → 调度决定谁运行
  → 并发访问共享数据产生竞态
  → 锁保证互斥，信号量/条件变量协调条件
  → 多资源等待可能死锁
  → IPC 让独立进程协作，仍然需要同步和例外处理
```

第一轮以“能解释”为目标；第二轮以“能画图、算题、写伪代码”为目标；第三轮闭卷做文末自测。

1. 操作系统结构、特权级与系统调用

对应：I1 第 13–20、24–34、41–50 页；I2 第 7–38、40–46 页。

1.1 必须能用自己的话解释

- **OS 的两面性：**向上提供抽象、接口和共用服务；向下管理硬件，实现共享、隔离和保护。CPU 对应执行流，内存对应地址空间，持久化存储对应文件等抽象。
- **系统设计是权衡：**性能、公平性、可靠性、安全、成本等目标可能冲突。“最快”不等于“适合所有场景”。
- **发展动机：**单道批处理减少人工交接；多道程序通过计算与 I/O 交叠提高利用率；分时系统通过时间片支持交互与公平共享。多道程序不要求多核。
- **启动链：**上电/复位 → 固件初始化必要硬件 → 加载并运行 Bootloader → 加载内核 → 初始化内核与服务。ROM 中的起始代码解决“谁先把 OS 装入内存”的问题。
- **用户态与内核态：**硬件特权级限制危险操作，内存保护限制地址访问。仅仅约定“应用不要乱写”不能形成保护。
- **库不是内核：**`printf`、`malloc` 通常先在用户态执行；需要 OS 服务时才通过相关系统调用进入内核。不能把所有 C 函数都当系统调用。

系统组成可按五组职责回忆：处理器管理（调度、同步）→ 内存管理（分配、保护、共享）→ I/O 管理（驱动、设备访问）→ 文件系统（名称与存储块、可靠性）→ 用户接口。不要只背名称，要能为每一组举出一个 OS 必须解决的问题。[I2，第 40–46 页]

1.2 异常分类和控制流

按课件 I2 的分类记忆；不同体系结构对 trap/exception 的总称可能不同，答题时先看题目语境。

事件	与当前指令关系	典型例子	处理后可能去向
硬件中断	异步，通常来自外部事件	时钟、设备完成	恢复被中断的执行；也可能调度其他任务
Fault	同步，当前指令遇到可修复问题	可修复的缺页	修复后重试故障指令；无法修复则终止
Trap	同步，主动触发或约定的控制转移	系统调用、调试陷入	按相应约定恢复，课件例子为下一条指令
Abort	严重、通常不可恢复	某些硬件错误	通常不能正常继续该程序

辨析：发生中断不等于一定切换进程；可屏蔽中断不等于所有异常都能关掉。缺页也不一定是程序错误，可能只是所需页面尚未就绪。

1.3 系统调用必须能画出的完整流程

用户程序调用库封装
→ 按 ABI 放置系统调用号与参数
→ 执行受控入口指令（如 RISC-V ecall、x86 syscall）
→ 进入内核入口，保存必要现场、使用内核栈
→ 检查调用号、权限和用户参数
→ 分派到具体服务
→ 返回结果/错误，恢复现场并返回用户态

参数可以放在寄存器中，也可通过“寄存器中的用户缓冲区地址”间接传入。内核不能直接信任这个地址，必须检查并安全访问用户内存。[I2，第 23–35 页]

ABI 规定二进制层面的调用约定，例如参数/返回值放哪些寄存器、哪些寄存器由谁保存；返回用户态也需要相应的受控返回机制。**辨析：**遵守 ABI 不等于能在任意 CPU 或任意 OS 上运行，二进制兼容还需要相应的指令集、平台 ABI、加载器与运行库支持。[I2，第 31–32 页]

比较	普通函数调用	系统调用
是否必须改变特权级	否	用户程序请求内核服务时是
跳转约束	按程序调用约定	经受控入口和参数检查
开销组成	调用/返回、保存必要寄存器	还包含陷入、保护检查及必要的状态管理
是否必然阻塞或切换进程	否	也不必然

1.4 架构比较题的答题骨架

结构	核心思想	优势	代价
宏内核	大量 OS 子系统在内核地址空间运行	内部调用直接，协作方便	故障隔离较弱，内核开发约束强
微内核	内核保留必要机制，更多服务移到用户态	模块化、故障隔离	更多 IPC 与保护边界穿越
LibOS / Exokernel 思路	将部分策略与服务放入应用关联的库	可定制，减少部分通用抽象开销	通用性、兼容性与资源协调更复杂

课件使用 Linux/BSD、Mach/MINIX 等例子。**小拓展：**实际系统常是混合设计；LibOS 并不意味着可以绕过所有安全隔离，也不意味着底层绝对不能共享资源。

会做的题：为什么用户不能直接修改页表或关闭中断？`malloc` 每次都需要陷入内核吗？一个 `read` 为何有时立即返回、有时导致阻塞？

2. 进程、地址空间与上下文切换

对应：P1 第 5–20、22–35 页。

2.1 进程的表示

进程 = 程序的一次动态执行 + 使用的资源 + 运行状态。程序文件是静态对象；同一可执行文件可以对应多个进程。

PCB 保存/关联进程标识、状态、调度信息、寄存器上下文、地址空间信息、打开文件等。它不是把整个进程内存逐字节塞进去的“大数组”。[P1，第 17–20 页]

区域	主要内容	例子
`.text`	指令	函数机器码
`.rodata`	只读数据	字符串常量
`.data`	已初始化的全局/静态数据	`static int a=3;`
`.bss`	零初始化的全局/静态数据	文件作用域的 `int a;`
堆	动态分配区域之一	`malloc` 返回的内存
栈	调用帧、自动变量等	一般的局部变量、返回信息

典型图中堆栈相向增长，但不是所有体系结构、分配器和运行时的普遍定律；大块分配还可能使用独立映射区域。[P1，第 8–15 页]

补上“源码如何成为进程”的链条：**编译 → 汇编 → 链接 → 加载**。ELF 的 section 偏向链接视图，segment 偏向装载视图，两者不能一一等同。[P1，第 9–12 页]

2.2 状态转换：必须闭卷画对

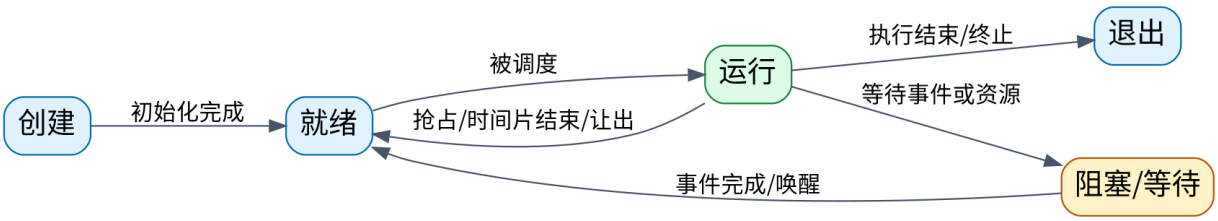


图 1: 进程主要状态转换

转换	原因
就绪 → 运行	被调度并获得 CPU
运行 → 就绪	时间片耗尽、被更高优先级任务抢占、主动让出
运行 → 阻塞	等待 I/O、锁或其他条件
阻塞 → 就绪	等待事件发生、资源可用、被唤醒
运行 → 退出	正常结束或被终止

易错：唤醒只是使其具备竞争 CPU 的资格，不意味着立即运行。等待 I/O 的进程不是“就绪但暂时没有分到 CPU”。`yield` 通常让出 CPU 后仍可运行，不能用它直接代替“等待某条件”的阻塞操作。

2.3 上下文切换究竟保存什么

保存当前执行流的 PC、SP、必要通用/状态寄存器等；恢复下一个执行流。切换不同地址空间时，还涉及相应的地址转换上下文。

- 不要求每次把整个内存写入磁盘。
- 不要求每次把 CPU 数据 Cache 全部清空。
- TLB 如何处理与硬件、地址空间标识和内核实现有关，不能和 Cache 混为一谈。
- 用户态进入内核态可能仍是同一个进程在执行；**模式切换不等于进程切换**。[P1，第 30–31 页；I2，第 27–28 页]

2.4 `fork / exec / wait / kill` 对比

API	对进程的作用	重点
`fork`	创建子进程	成功时父进程得到子 PID，子进程得到 0；后续执行分别继续
`exec`	替换当前进程的程序映像	成功不返回；不因 exec 本身新增一个 PID
`wait/waitpid`	等待并回收子进程退出状态	等待者可能阻塞；不只是“让子进程先执行”
`kill`	给指定进程发送信号	名字叫 kill，但并非所有信号都终止进程

辨析： 课件 P1 第 34 页把 fork 与写时复制放在一起。准确理解为：父子进程具有逻辑上独立的地址空间，初始物理页面可按 COW 共享；一方修改私有页面时分离。它不是线程共享内存那种“改变量后另一方自动看到”。

小拓展： 若子进程退出后父进程尚未 wait，通常还需保留少量退出状态供回收，这就是理解僵尸进程的入口。不要把它理解成子进程仍在继续计算。

会做的题： 两进程中同一个虚拟地址一定对应同一物理地址吗？fork 后修改普通全局变量会影响父进程吗？若连续执行 n 次 fork 且所有分支都继续、都成功，一共有 2^n 个进程。

3. 线程模型、并发与并行

对应：P2 第 5–15、17–26、28–35 页。

3.1 进程与线程的比较

项目	进程	同一进程内的线程
地址空间	通常独立，除非显式共享	共享整个进程地址空间
资源	关联文件、内存、权限等资源	通常共享所在进程的这些资源
执行状态	可包含一个或多个执行流	各有 PC、SP、寄存器上下文和栈
主要抽象	资源与隔离边界	可调度执行流

TCB 记录线程状态、寄存器上下文、栈等信息。**线程有各自的栈，不等于别的线程在硬件上绝对不能访问该栈**：它们共享地址空间，若有有效地址仍可能访问；还要考虑对象生命周期。

线程切换通常不必更换同一进程的地址空间，代价一般小于切换不同进程，但仍然有寄存器、内核调度、Cache 工作集等开销。

3.2 线程映射模型

模型	谁管理用户执行流	多核并行与阻塞影响
多对一 M:1	用户态线程库，映射到一个内核调度实体	不能同时占多个核；阻塞该实体可能阻塞整组
一对一 1:1	每个用户线程对应内核可调度实体	可多核并行；一个阻塞不必阻塞其他线程
多对多 M:N	用户运行时将多个执行流映射到多个内核实体	并行度受 N 和硬件限制；需协调两层调度

课件以 LWP 连接用户线程与内核调度实体。术语随系统不同，不要把 LWP 当成永远固定的第三种神秘执行单位。[P2，第 17–26 页]

重点辨析：“内核级线程”指由内核可见和调度的线程，不等于“线程的业务代码只能在内核态执行”。课件第 19 页的“内核工作线程”是另一个具体概念。Linux `pthread` 的用户代码可以在用户态运行，同时具有对应的内核调度实体。

3.3 并发、并行与线程 API

- **并发：**多个任务在一段时间内交错推进；单核也能有并发。
- **并行：**同一时刻实际同时执行；需要多个执行资源。CPU 与设备 I/O 也能交叠。
- **过程调用：**同一线程里进入新的调用帧，本身不创建独立调度实体。
- `pthread_create` 创建线程；`pthread_join` 等待目标线程结束。join 不是“等所有线程到同一阶段后再一起继续”的可复用屏障。

小拓展：Amdahl 定律。串行比例为 s，其余部分理想地分到 N 个处理器：

$$S(N) \le \frac{1}{s + (1-s)/N}.$$

例如 $s = 0.2, N = 4$，加速比最多 2.5，处理器无限多时最多 5。实际还要减去同步、调度和数据搬运等开销。这回应 P2 第 34 页的“为什么增加伙伴不能总得到线性加速”。

会做的题：同一进程两个线程分别修改全局变量与栈上的局部变量，哪些会共享？用户级线程是否必然无法并行？回答后一个问题必须先说明 M:1 还是 M:N。

4. CPU 调度：比较、计算与权衡

对应：P3 第 3–7、9–31 页；第 32–33 页是课件自身的 BSD/Linux 扩展。

4.1 先把三个“时间”分开

设到达时刻 A_i、首次得到 CPU 的时刻 S_i、完成时刻 C_i、CPU 总执行时间 B_i：

$$\text{周转时间} T_i = C_i - A_i, \qquad \text{响应时间} R_i = S_i - A_i.$$

无 I/O 等其他阻塞、只有 CPU 执行和就绪等待时：

$$\text{等待时间} W_i = T_i - B_i.$$

若题目含 I/O，等待时间是就绪队列中等待的总和，不能把所有 I/O 阻塞也算成就绪等待。响应时间只数到首次得到响应/服务的约定时刻；调度题通常取首次获得 CPU。课件第 9 页把“响应时间（等待时间）”放在一起，计算题中应明确区分。

抢占式调度可因时间片耗尽或更高优先级任务到达而切换；非抢占式通常等当前任务完成、阻塞或主动让出。不要把“进程可以被抢占”与“内核代码任何位置都能被抢占”等同。

4.2 算法对照表

算法	选择依据	核心收益	主要问题
FCFS	就绪队列到达顺序，非抢占	简单	长作业在前的护航效应，短作业等待长
STCF（课件）	最短预计执行时间，非抢占	在适当假设下减小平均等待/周转	需预测执行时长，短任务持续到达会使长任务饥饿
SRTCF（课件）	最短剩余时间，抢占	新短任务可更早完成	需预测剩余时长，可能饥饿
RR	按队列轮转，每次最多一个时间片	交互响应、公平轮流	时间片短则切换开销大，长则趋近 FCFS
VRR	给 I/O 返回任务辅助队列	对 I/O 型任务更友好	需管理队列和剩余配额

算法	选择依据	核心收益	主要问题
MLQ	任务分固定类别队列	不同类别采用不同策略	低优先级队列可能饥饿
MLFQ	根据历史行为升降级	不必预知执行时长，兼顾交互与计算	需设时间配额、提升等规则
FSS / 彩票	用户或组的份额	按比例共享 CPU	短期比例可能波动，需考虑统计公平
EDF	最早绝对截止时间，通常抢占	面向实时截止要求	需满足负载和可调度性前提

术语提醒：本课 P3 第 14 页的 STCF 是非抢占式，接近其他教材的 SJF；第 15 页的 SRTCF 接近 SRTF。有些书把 STCF 用来指抢占式算法，考试时按本课定义并写清“是否抢占”。

FCFS 对已有有限前驱且会完成的任务通常不会造成无限期饥饿，不要把“等很久”直接等同“饥饿”。

4.3 课件例题必须重算：RR，时间片 20

四任务同时在 0 到达，按 P1、P2、P3、P4 入队，CPU 时间分别为 53、8、68、24，不计切换开销。[P3，第 18–20 页]

```
0-20 P1 | 20-28 P2 | 28-48 P3 | 48-68 P4 | 68-88 P1
88-108 P3 | 108-112 P4 | 112-125 P1 | 125-145 P3 | 145-153 P3
```

任务	完成 C_i	周转 T_i	等待 W_i	首次响应 R_i
P1	125	125	72	0
P2	28	28	20	20
P3	153	153	85	28
P4	112	112	88	48
平均	—	104.5	66.25	24

不要把平均等待 66.25 写成平均响应，也不要把某任务分多次运行的所有结束时刻相加。

4.4 迁移练习：有新任务到达时怎么重排

任务为 P1=(到达 0，执行 8)、P2=(1，4)、P3=(2，2)、P4=(3，1)。剩余时间相同时保留当前任务。

- 非抢占 STCF：`P1[0,8] → P4[8,9] → P3[9,11] → P2[11,15]`。
- 抢占 SRTCF：`P1[0,1] → P2[1,2] → P3[2,4] → P4[4,5] → P2[5,8] → P1[8,15]`。
- 两者平均等待时间分别为 5.5 和 2.75。

做题步骤：标出到达事件 → 只看已就绪任务 → 更新剩余时间与队列 → 画时间轴 → 求完成时刻 → 套公式。RR 还须按题意明确“新到达与时间片结束恰好同时”时的入队次序。

4.5 能解释的几个权衡

- **优先级反转：**低优先级 L 持锁，高优先级 H 等锁，中优先级 M 抢占 L，使 H 间接被 M 延迟。优先级继承暂时提高持锁者 L，使其尽快释放锁；它不能普遍消除死锁。[P3，第 22 页；D，第 33 页]
- **MLFQ 与 MLQ：**关键不是“都有多个队列”，而是任务能否按反馈在队列间移动。累计使用 CPU 配额后降级、长时间等待后提升，防止靠频繁主动让出 CPU 钻规则空子。[P3，第 23 页]
- **份额与优先级：**“先运行”不等于“长期应分到多少”。用户 A 两个任务、B 一个任务，要两用户平分资源，可设任务份额 25%、25%、50%，而非简单三任务各 1/3。[P3，第 24–26 页]

彩票调度给某任务至少一张票，使其具有正的被选概率；长期比例可接近份额，但这不是确定的有限等待上界，更不能直接保证硬实时截止时间。

小拓展：时间片开销。若每运行 q 时间有一次开销 c，简化开销占比为 $c/(q+c)$。要不超过 1%，需 $q \geq 99c$；不能只说时间片越小越好。

4.6 实时调度与课件中的扩展

硬实时要求必须满足截止时间，软实时允许一定违约；EDF 按截止时间，不是按剩余计算量排序。[P3，第 28–31 页]

$$U = \sum_i C_i/T_i.$$

课件例子 $1/4 + 2/5 + 2/7 = 131/140 < 1$。**辨析：**将 $U \leq 1$ 作为充分判断，需采用单处理器、独立、可抢占、截止时间等于周期、忽略切换和资源阻塞等理想 EDF 模型；它不是对任意固定优先级策略或任意截止时间的万能结论。

课件第 32–33 页的 BSD 动态优先级和 Linux CFS 先掌握思想：近期 CPU 使用影响优先级；CFS 以权重归一化的虚拟运行时间比较服务量。nice 范围为 -20 到 19，nice 越大通常权重越低、同样实际运行时间累计的虚拟时间越快，可概括为虚拟时间增量与权重成反比。复习以课件模型为准，不扩展追踪每个内核版本的实现。

5. 临界区、锁与原子操作

对应：L1 第 6–18、20–33、35–44 页；L+ 第 2–9 页。

5.1 从错误交错推出“为什么要锁”

两个线程执行 `counter++`，可抽象为读取、加一、写回。若都读到 0，再分别写 1，结果是 1 而不是 2。分配 PID 的例子同理，可能得到重复 PID。[L1，第 6–8 页]

临界资源是被保护的共享对象；**临界区**是访问它、需要互斥执行的代码段；**竞态条件**是正确性受不受控交错影响；**原子性**是相对其他并发观察者表现为不可分割。

原子性不等于“整个期间硬件绝不接收中断”。锁保护的临界区可以被暂停，但其他遵守同一锁协议的线程不能同时进入。所有相关访问必须遵守一致的保护规则，不能一半加锁、一半裸访问。

5.2 软件互斥的四类尝试

方案	关键步骤	为什么不够
严格轮流	`while(turn != i)`	对方不进入时也可能阻止自己，违反进展性
先检查、后举旗	看对方 flag，再设自己 flag	二者都可能先检查到“空闲”，同时进入
先举旗、后检查	自己 flag=true，再等对方 flag=false	二者同时举旗，可能互相等待
Peterson	flag 表示意愿，turn 在冲突时让步	在经典原子读写、顺序一致模型下满足二线程互斥与进展

Peterson 必须会默写并解释每行：[L1，第 25–26 页]

```
acquire(i):
    j = 1 - i
    flag[i] = true
    turn = j
    while flag[j] and turn == j:
        spin

release(i):
    flag[i] = false
```

两者都想进入时，最后一次给 turn 赋值者让对方先进入。不能把循环条件误写成 OR。

小拓展： Peterson 的证明依赖内存模型，不是“天生只支持单核”。现代编译器重排和弱内存序会破坏普通变量版本的假设；课程伪代码不能原封不动当作无同步的 C/C++ 多线程代码。真实实现应使用相应原子操作与内存序保证。

5.3 硬件原子操作与自旋锁

TAS 将“返回旧值”和“设置新值”作为一次原子读改写。锁初值为 0：[L1，第 29–33 页]

```
acquire():
    while TAS(lock, 1) == 1:
        spin
release():
    atomic_store_release(lock, 0)
```

TAS 不是普通的 `if(lock==0) lock=1`。LL/SC 是“建立保留 → 计算 → 条件存储，失败重试”；Fetch-and-Add 原子地加值并返回旧值。需要保证原子读改写及适当的可见性，不能仅靠 C 的 `volatile` 实现锁。

5.4 自旋、睡眠与关中断

方式	获取不到资源时	适合情形	主要限制
自旋锁	持续占用 CPU 检查	持锁时间短，持有者能继续执行	长等待浪费 CPU；普通内核自旋临界区不能随意睡眠
阻塞互斥锁	排入等待队列并阻塞	等待可能较长	阻塞和唤醒有调度开销，须正确处理竞态
本核关中断	暂时不响应可屏蔽中断	很短的内核关键操作	不阻止其他核，时间长会增加中断延迟

必须能分析的课件代码：关中断后一直自旋，可能让单核持锁者永远没机会继续；先开中断再入等待队列，可能丢失唤醒。[L1，第 35–43 页]

典型丢失唤醒交错：等待者发现“锁忙”→ 释放者解锁并发现没人等待 → 等待者才入队并睡眠。以后可能没有任何人再唤醒它。

正确抽象是：**检查条件、登记等待、释放保护并阻塞**之间要有与唤醒方匹配的原子协议；不能把“入队”和随后的 `sleep` 随意拼接。教材的逐步尝试用于分析竞态，不是可直接复制的完整多核锁实现。

小拓展：futex。无争用时先在用户态做原子操作，有争用才进入内核等待/唤醒；不是每次成功获取 pthread mutex 都必然做系统调用。[L1，第 44 页]

5.5 读写锁和 RCU：这是课件已有的扩展

读读允许、读写互斥、写写互斥。读者优先可能使写者饥饿；写者优先若写请求持续到达，也可能延迟读者。[L+，第 3–4 页]

RCU 的主线：**读者访问当前版本 → 更新者准备新版本并安全发布 → 等旧读侧临界区结束 → 回收旧版本**。适合读多写少，不应把“指针已替换”理解成“旧对象可以立即 free”。RCU 不自动解决所有写者之间的竞争。[L+，第 5–9 页]

6. 死锁、安全状态与银行家算法

对应：D 第 3–9、12–20、21–34 页。

6.1 四个必要条件必须连同含义一起记

1. **互斥：**某些资源不能同时被多个任务占用。
2. **占有且等待：**持有资源的同时等待其他资源。
3. **不可抢占：**所需资源不能被任意强行取走。
4. **环路等待：**形成首尾相接的等待链。

例：A 持 L1 等 L2；B 持 L2 等 L1。用图或两列时间线写清“已持有”和“还请求”，不能只说“同时加锁所以死锁”。

辨析：在每类资源只有一个实例的标准资源分配图中，环路可判定死锁；有多个实例时，有环不一定死锁。饥饿是某任务长期得不到服务；死锁是集合内互相等待；活锁则可能不断动作、退让和重试，却不推进任务。

6.2 四类处理策略

策略	核心	示例/代价
忽略	把恢复交给应用或人工	杀进程、重启，简单但损失工作
检测并恢复	允许发生，再检测和解除	查等待关系、终止/回滚或可行的资源抢占
动态避免	每次分配前检查未来完成的可能性	银行家算法，需要最大需求等信息
静态预防	事先破坏必要条件	统一锁顺序、一次申请全部资源等，可能降低并发

注意：CPU 可以抢占，不代表任意已持有的锁和设备状态都可以安全抢占。统一按锁编号递增申请是破坏环路等待；并非破坏互斥。[D，第 30–34 页]

6.3 安全、非安全和已死锁

安全状态是存在一种顺序，使每个任务都能在满足其剩余最大需求后完成并释放资源。安全序列是完成可行性的证明，不要求 OS 此后真的只按这一顺序调度 CPU。

- 安全状态存在一条保证完成的分配路径；后续仍需采用安全分配，不能理解为“此后怎么分都不会死锁”。
- 非安全状态没有这种最坏需求保证，但可能因任务未请求全部最大需求而完成。
- 因此 **非安全不等于已经死锁**；银行家算法比“只拒绝已经死锁”更保守。

6.4 安全性算法与请求算法：必须写对

$$Need = Max - Allocation.$$

所有向量比较均逐分量进行。Need 是“剩余最大需求”，不是“本次实际请求”。

```
Work = Available
Finish[i] = false  for every i
sequence = []
while 能找到 Finish[i]==false 且 Need[i] <= Work:
    Work += Allocation[i]
    Finish[i] = true
    sequence.append(i)
```

若所有 `Finish` 为 `true`，则安全；否则不安全

辨析： D 第 16 页的初始化写了 `Finish=true`，但后面的寻找条件是 `Finish=false`；安全性算法的初始完成标记应为 **false**。模拟完成后加回原 Allocation，不要重复把 Need 当成额外归还资源。

收到 $Request_i$ 后：[D，第 18–20 页]

1. 检查 $Request_i \le Need_i$，否则超出声明需求。
2. 检查 $Request_i \le Available$，否则等待。
3. 试分配：Available 减 Request，Allocation 加 Request，Need 减 Request。
4. 对试分配状态做安全性检查；安全则接受，不安全则**回滚三项更新**并推迟/拒绝本次分配。

6.5 课件矩阵例题的完整安全检测

按 D 第 21 页，资源总量为 $(9,3,6)$，Available 为 $(0,1,1)$：

任务	Allocation	Max	Need
T1	(1,0,0)	(3,2,2)	(2,2,2)
T2	(6,1,2)	(6,1,3)	(0,0,1)
T3	(2,1,1)	(3,1,4)	(1,0,3)
T4	(0,0,2)	(4,2,2)	(4,2,0)

一条安全序列及 Work 的变化为：

$$(0,1,1) \xrightarrow{T2} (6,2,3) \xrightarrow{T1} (7,2,3) \xrightarrow{T3} (9,3,4) \xrightarrow{T4} (9,3,6).$$

迁移题： 若此时 T1 请求 $(0,0,1)$，虽然不超 Need 且资源够，试分配后 Work=$(0,1,0)$，没有任何任务满足 Need≤Work，故不能立即接受。这个例子说明“有空闲资源”不等于“可以安全分配”。

7. 信号量、条件变量、管程与屏障

对应：S 第 3–17、19–38、40–44 页。

7.1 先区分几类机制

机制	回答的问题	关键语义
锁	谁能访问共享数据?	互斥；所有相关访问遵守同一协议
信号量	有几个资源/许可可拿?	P 获取许可，V 释放/增加许可，可保留计数
条件变量	某状态还不满足时如何等待?	结合锁与共享谓词；通知不累积为许可
管程	如何封装共享状态、互斥与等待?	一种组织结构，不只是某个函数名
屏障	一组线程是否都到达本阶段?	到齐后才允许越过同步点

锁本身只能保护缓冲区操作，不能自动表达“空时消费者必须等，满时生产者必须等”。[S，第 3–4 页]

7.2 信号量的两个常用用途

- **互斥：**初值 1，临界区前 P，结束后 V。
- **先后同步：**初值 0，A 做完后 V，B 先 P 再做依赖 A 的工作。即使 A 的 V 先发生，许可仍可保留给 B。

课件存在两种描述：非负计数模型中，P 在没有许可时等待；阻塞型实现中先减一，值为负表示等待者数量的相反数。这不是两个相互矛盾的规则。增减、队列操作和阻塞/唤醒必须满足原子协议。[S，第 5–6 页]

7.3 必须默写：有界生产者—消费者

容量为 N，初值为 `empty=N, full=0, mutex=1`。以下均为算法伪代码：

```
producer:                    consumer:
    item = produce()             P(full)
    P(empty)                     P(mutex)
    P(mutex)                     item = remove()
    insert(item)                 V(mutex)
    V(mutex)                     V(empty)
    V(full)                      consume(item)
```

为什么先 P(empty/full)，后 P(mutex)？若消费者先持 mutex，再在空缓冲区上 P(full)，就会阻塞；生产者又必须取得 mutex 才能放入数据，形成互相等待。

`empty/full` 控制数量条件，`mutex` 保护队列内部一致性；多生产者/多消费者时不能省略后者。[S，第 7–10 页]

7.4 哲学家与读者—写者

哲学家：所有人都先拿左叉再拿右叉可能死锁。课程给出大锁串行化和奇偶改变取叉顺序两种修复；统一按叉子编号从小到大获取也可破坏环路等待。大锁正确但降低并行性。[S，第 11–14 页]

辨析：无死锁不自动保证每个人无饥饿。还要考虑锁/信号量是否公平、是否持续被别人抢先。

读者优先模板：`Rcount=0, CountMutex=1, WriteMutex=1`。[S，第 15–17 页]

```
reader:                          writer:
    P(CountMutex)                    P(WriteMutex)
    if Rcount == 0:                  write()
        P(WriteMutex)                V(WriteMutex)
    Rcount++
    V(CountMutex)
    read()
    P(CountMutex)
    Rcount--
    if Rcount == 0:
        V(WriteMutex)
    V(CountMutex)
```

第一位读者拦住写者，最后一位读者放行写者；计数更新本身也要互斥。后来的读者可以加入已有读者群，因此写者可能饥饿。课程第 37–38 页通过 AW/AR/WW/WR 的等待谓词实现对写者友好的管程版本。

7.5 条件变量必须记住的三个动作

`wait(cv, mutex)` 应视为：**原子地释放 mutex 并进入等待 → 被通知后重新竞争 mutex → 成功持锁后才返回**。

- 谓词（例如 `count>0`）存在共享数据中，条件变量本身不是一个保存真假值的布尔量。
- `signal` 无等待者时通常不保留通知；这与信号量 V 的计数不同。
- `signal` 不意味着调用者释放锁，也不意味着被唤醒者立即拥有锁。

Mesa 风格下的模板：[S，第 19–24、30–34 页]

```
put(item):                          get():
    lock(m)                             lock(m)
    while count == N:                   while count == 0:
        wait(not_full, m)                   wait(not_empty, m)
    insert(item); count++               item = remove(); count--
    signal(not_empty)                   signal(not_full)
    unlock(m)                           unlock(m)
                                        return item
```

这里特意用 not_full / not_empty 命名，表示“等待什么变为可用”，避免将课件中 full、empty 的名称与谓词含义混淆。

7.6 为什么用 while：用交错过程回答

消费者 C1 因为空而等待；生产者放入一件并通知 C1；在 C1 重新获得锁之前，另一消费者 C2 先取走了它。C1 醒来时又是空缓冲区。

因此 wait 返回只意味着“可以重新检查条件”，不保证谓词仍成立；还可能出现伪唤醒。用 if 可能从空队列取数据，用 while 才能再次等待。

管程语义	signal 后谁继续执行	对条件判断的影响
Hoare	被唤醒者立即接手，发送者暂挂	理想交接语义下，被通知条件可立即使用
Mesa	发送者继续，被唤醒者以后竞争锁	必须循环重查谓词
Hansen	发送者在退出管程处通知	signal 位置受到限制

Java、pthread 条件变量和常见 C++ 条件变量按 Mesa 风格理解。本表不意味着现实 API 中可以随便用 if 代替 while。

7.7 屏障：到齐，再继续

课件 S 第 40–42 页的关键是“固定 N 个参与者都到达同一同步点”。不要混用两种计数：**已到达 arrived 初值 0**；**剩余未到达 remaining 初值 N**。

小拓展：可复用屏障。用轮次避免把上一轮唤醒与下一轮到达混在一起：

```
shared: N, arrived=0, generation=0, mutex m, condition cv
barrier_wait():
    lock(m)
    my_generation = generation
    arrived++
    if arrived == N:
        arrived = 0
        generation++
        broadcast(cv)
    else:
        while generation == my_generation:
            wait(cv, m)
    unlock(m)
```

前提是固定参与者每轮都到达且没有中途永久退出。若只把 count 重置为 0，而旧等待者仍等 `count==N`，可能让它们错过条件。

8. 进程间通信：机制选择与异常处理

对应：C 第 3–12、14–29、31–40 页。

8.1 五种机制的选择题

机制	适合传什么	核心特点	必须注意
信号	事件通知	信息少，处理方式有默认、捕获、忽略、屏蔽等	不用于大量数据；处理函数有限制
管道/FIFO	本机字节流	经内核缓冲，一端写一端读	字节流不自带应用消息边界；正确关闭端点
消息队列	独立消息	保留消息结构，可多对多	类型、队列容量、阻塞/非阻塞与生命周期
共享内存	大量本机共享数据	映射相同物理区域，减少反复拷贝	需要另配同步，建立映射本身不是加锁
Socket	本机或跨机器通信	TCP/UDP 等通信端点抽象	连接/报文语义、流量与失败处理

无名管道通常通过 fork 继承端点用于亲缘进程；FIFO 通过路径让无亲缘进程找到它。**辨析：**这不是说管道机制只能有一个写者和一个读者；多个端点持有者仍要处理写入原子性和数据边界。

消息队列以消息为单位，不应理解成和管道完全相同的无结构字节流；SysV 队列按类型选择接收时，也不能笼统认为所有接收都严格按全局 FIFO。

课件 API 配对：消息队列用 `msgget / msgsnd / msgrcv / msgctl`；共享内存用 `shmget / shmat / shmdt / shmctl`。`shmat` 是映射/附接，`shmdt` 是解除本进程的映射，不能直接等同于删除所有进程共享的对象。[C，第 21–26 页]

8.2 信号相关辨析

- `kill(pid, SIGTERM)` 是请求终止；处理程序可以做清理。
- `SIGKILL`、`SIGSTOP` 不能被捕获、忽略或屏蔽。
- **忽略**是处置方式，**屏蔽**是暂时不交付，可能留下 pending 状态；二者不同。
- 不要把硬件中断、进程信号、条件变量 signal、信号量 V 当成同一种机制。

小拓展：C 第 9–11 页的演示用处理函数打印信息，帮助观察控制流。真实 POSIX 信号处理器中，不应随意调用 `printf`、普通加锁等非异步信号安全操作；常见做法是仅设置适当的标志，或通过安全机制交给正常执行流处理。对同步故障信号也不能靠“忽略后继续”可靠修复错误。

8.3 管道必须会分析的三个条件

`pipe(fd)`：`fd[0]` 是读端，`fd[1]` 是写端。fork 后两进程都继承端点引用，应关闭各自不用的那一端。[C，第 14–18 页]

- 缓冲区空、但仍有写端打开：阻塞读通常等待数据。
- 缓冲区已空、且所有写端都关闭：读返回 0，即 EOF。若仍有未读数据，先读完数据再见 EOF。

- 没有读者时写：典型 POSIX 行为涉及 SIGPIPE；若未被默认终止，可通过 EPIPE 等错误处理。

典型题：父进程明明已经退出，子进程为何一直读不到 EOF？先检查子进程自己是否仍保留一个写端。管道容量不应死记成所有系统都固定 64 KiB，也别把容量与一次写入的原子性界限混淆。

8.4 Socket 调用链与读写语义

TCP 服务端：`socket → bind → listen → accept → read/write → close`。

TCP 客户端：`socket → connect → read/write → close`。

accept 返回用于这次连接的新描述符，监听描述符仍可继续接受连接。UDP 不使用同一套 TCP 建连流程。[C，第 28–29 页]

小拓展：TCP 是字节流，“一次 write”不保证对方“一次 read”恰好取得相同长度。需要定长、分隔符或长度字段等应用协议处理消息边界；非阻塞也不等于异步，调用可能只是立即返回“暂时不能完成”。

8.5 IPC 设计题答题顺序

先确定数据量/消息边界、同机或跨机、通信关系；再确定缓冲、阻塞方式和同步；最后处理退出、超时、重传与重复。[C，第 31–40 页]

问题	必要处理
接收者或发送者退出	关闭通知、错误返回或超时，不能无限等一个不会再出现的事件
发送/接收端暂时来不及处理	缓冲与背压；按需要采用阻塞或重试
消息丢失/ACK 丢失	ACK + 超时重传，但必须考虑重复
收到重复消息	序列号、去重或幂等处理，不能只说“再执行一次”
消息损坏	校验检测，再重传或用纠错机制

TCP/UDP、管道或消息队列的具体保证不同，不能把课件中的一般网络丢包模型无条件套到所有本机 IPC 上。

9. 最容易混淆的概念：闭卷对照

一组概念	一句话区分
模式切换 / 上下文切换	进入内核未必更换执行任务；换任务需要保存与恢复执行上下文
就绪 / 阻塞	前者缺 CPU，后者缺事件或资源

一组概念	一句话区分
进程 / 程序 / 线程	动态资源上下文 / 静态代码数据 / 独立执行流
并发 / 并行	交错推进 / 实际同时执行
fork / exec	新建进程 / 替换当前映像
等待 / 响应 / 周转	就绪队列总等待 / 首次服务前等待 / 到达到完成
死锁 / 饥饿 / 活锁	互相等待 / 个体一直得不到服务 / 有动作但无有效进展
预防 / 避免	结构上破坏必要条件 / 按状态审慎分配
锁 / 信号量 / 条件变量	互斥所有权 / 许可计数 / 围绕共享谓词等待
signal 与 V	条件变量通知不累积许可；信号量 V 可以增加许可
`waitpid` / 条件变量 wait / join / barrier	子进程退出回收 / 条件等待 / 目标线程退出 / 一组线程阶段会合
共享内存 / 自动同步	共享存储不等于访问有序，仍要锁、原子或其他协议
线程栈独立 / 地址空间隔离	各自使用不同栈，不等于栈在彼此不可达的地址空间
优先级 / 份额	紧急先后 / 长期分配比例

10. 自测题与核对答案

先遮住后半部分答案。达到“说出机制，并给出反例或交错顺序”才算掌握。

10.1 闭卷题

1. 为什么只靠普通函数调用不能代替受控系统调用入口?
2. 一次不阻塞的系统调用返回后，是否必然换了 PID?
3. 写出 `sleep` 从开始等待到再次执行的状态路径。
4. fork 后父子普通全局变量的虚拟地址相同，能否据此证明变量共享?
5. 一个 M:1 用户线程执行阻塞系统调用，其他线程为什么可能停住? 1:1 呢?
6. 三任务同时到达，CPU 时间为 24、3、3。FCFS 按此顺序的平均周转和平均等待是多少?
7. 若切换开销为 0.01 ms，要使简化开销比例不超过 1%，时间片至少多少?
8. 时间片缩短是否一定同时改善平均响应、等待、周转与吞吐?
9. 两个 flag 都“先检查后设置”为什么不能互斥? 写出一种交错。
10. 为什么仅关闭本 CPU 的中断不能保护多核共享计数器?
11. 生产者—消费者中，将 `P(mutex)` 移到 `P(full)` 前，会造成什么问题?
12. 条件变量 signal 先发生，之后才有人 wait，这个人能自动消费那次通知吗?
13. 用 if 而非 while 检查缓冲区状态，给出一个失败过程。
14. 有环的多实例资源分配图能否直接断定死锁? 非安全是否一定已死锁?

15. 银行家算法的试分配失败后，哪三项状态必须回滚?
16. 第 6.5 节的 T1 请求 (0,0,1) 是否应立即批准?
17. 为什么指针已经替换后仍不能立即释放 RCU 旧版本?
18. 什么情况下管道 read 返回 0？还有数据没读完时也立即返回 0 吗?
19. 将共享内存映射给两个进程后，为什么还需要同步?
20. ACK 丢失导致消息重传，接收方如何避免重复扣款?

10.2 答案要点

1. 需要硬件保护边界、受控入口、特权切换和参数校验，不能由不受信任应用任意跳到内核私有实现。
2. 不会因系统调用本身必然换进程；例如 getpid 可以在同一任务中处理并返回。
3. 运行 → 阻塞 → 定时事件使其就绪 → 调度后运行。
4. 不能，独立地址空间与 COW 可使虚拟地址相同但后续写入分离。
5. M:1 只有一个承载执行实体，该实体被阻塞会影响整组；1:1 可调度其他就绪线程。
6. 完成时刻 24、27、30，平均周转 27；等待 0、24、27，平均等待 17。
7. $q \geq 99c = 0.99$ ms。
8. 不一定；切换开销、任务构成与各指标目标不同。
9. A 读到 flag[B]=false，B 也读到 flag[A]=false，之后都设置自己 flag 并进入。
10. 其他核仍能同时执行；需要跨核原子与同步机制。
11. 空缓冲区消费者持 mutex 等 full，生产者又等 mutex，无法产生 full。
12. 不能；但共享谓词可能已改变，正确程序应持锁检查谓词，已满足就不必 wait。
13. C1 被通知后尚未持锁，C2 先消费唯一数据，C1 返回时条件不成立；或伪唤醒。
14. 都不能；多实例有环不充分，非安全是缺少完成保证。
15. Available、Allocation[i]、Need[i]。
16. 不批准；试分配后 Work=(0,1,0)，无任务可假想完成。
17. 旧读者可能仍持有旧指针，需要等待相应读侧临界区退出/宽限期。
18. 缓冲区为空且所有写端引用关闭；未读数据应先返回给读者。
19. 映射只让双方访问同一存储，不建立复合操作原子性、访问顺序或条件等待协议。
20. 消息带唯一请求标识，接收方记录处理结果并去重，或设计幂等操作；仅重发 ACK 不能代替业务去重。

11. 三轮复习安排与考前检查

11.1 按掌握程度推进

第一轮：搭主线。先看第 1–3 节，把系统调用、进程状态、线程共享关系串起来。每个概念尝试用一句定义、一个例子、一个反例表述。

第二轮：做机制题。按第 4、5、7、6、8 节复习。调度要画时间轴，锁和条件变量要写交错过程，银行家要列 Work/Need 表，IPC 要讲清等待与退出条件。

第三轮：闭卷输出。不看模板写出 Peterson、生产者—消费者的两种实现、读者优先、银行家请求流程；完成第 10 节并回到对应课件核对。

若时间很紧，优先完成：**状态转换图 → 调度公式与 RR 例题 → P/V 和条件变量模板 → 银行家矩阵 → IPC 机制比较**。读写锁、RCU、CFS 等不能完全丢掉，先达到能说清目标和限制即可。

11.2 考前能独立完成的清单

- 不看课件画出系统调用流程和进程主要状态转换。
- 给定代码，指出数据、堆、栈和共享关系，并解释 fork/exec 的区别。
- 调度题不混淆时间指标，能处理到达、抢占与时间片边界。
- 对失败的锁实现写出具体竞态交错，而不只说“不是原子操作”。
- 知道哪些同步操作会释放锁，哪些只通知，哪些会保留计数。
- 对信号量给出名称、初值和每个 P/V 的含义。
- 银行家算法从 `Finish=false` 开始，试分配不安全时完整回滚。
- 区分“正确互斥”“无死锁”“无饥饿”，不把三者当同一个保证。
- 说清进程信号与同步 signal 的区别，以及管道 EOF 的条件。
- 设计 IPC 方案时主动检查同步、容量、关闭、超时和重复处理。

最有效的自检不是“这页我看过”，而是“合上资料后，我能把机制和一个失败反例重新写出来”。